

1. Capability and Authority

Astra can perform consequential actions through tools. Capability answers whether the agent can do something. Authority answers whether it should do it under the user's instruction. Confusing the two is the central permissions error.

A connected account or a writable workspace is not blanket authorization. The task determines the permitted action.

2. Sandbox

Codex sandboxing limits what commands or processes can access. Approval policy governs when the agent must request permission beyond the current boundary. These controls are useful because they make authority partly mechanical rather than relying entirely on prose.

The safest useful configuration is not always read-only. Engineering work often needs workspace writes and tests. The goal is least authority consistent with productive work.

3. A Boundary Taxonomy

Low-consequence reversible actions: read files, search code, inspect public docs, run local tests.

Workspace changes: edit tracked project files, generate build artifacts, create local branches.

External side effects: send messages, publish content, create or modify remote resources, change tickets.

Financial/account effects: purchases, subscriptions, account settings, credentials.

Destructive effects: delete data, force-push shared history, drop production state.

The latter categories need progressively clearer authorization. Exact policy depends on context, but broad capability should never silently collapse these distinctions.

4. Auto-Review and Monitoring

OpenAI describes additional monitoring around Astra for potential cases where agent behavior may not match the user's intent. Codex can also use review and approval mechanisms before or after changes. These protections are defense in depth, not replacements for a clear task boundary.

An agent can satisfy every sandbox rule and still implement the wrong feature. Safety controls limit damage classes; task verification establishes correctness.

5. The Convenience Objection

Frequent approval prompts can destroy the value of autonomous work. The answer is not "approve everything." It is to pre-authorize a coherent class of actions: inspect and edit within this repository, install project-local dependencies, run tests, create local commits, but ask before network publication, destructive migration, or changing accounts.

This converts dozens of micro-approvals into one meaningful boundary.

6. Trusted Repositories and Hidden Instructions

Project configuration itself can influence what Codex does. Treat unfamiliar repositories as potentially adversarial instruction environments. Inspect project guidance before granting broad permissions. Do not let a repository file redefine what external actions the user authorized.

The same reasoning applies to skills and plugins: they extend behavior and tools, so audit them according to consequence, not popularity.

7. Security Work

OpenAI positions Astra as unusually capable in cybersecurity and has additional safeguards for advanced capability. For ordinary users, the relevant lesson is narrower: security tasks should remain explicitly authorized, scoped to systems the user is permitted to test, and verified carefully. A powerful model increases the importance of boundaries.

8. Permission as Design

A good autonomous workflow specifies authority before execution begins. This reduces interruptions and prevents the agent from inferring permission from technical access. The ideal boundary is simple enough to remember and specific enough to enforce.

References: OpenAI, Codex security; OpenAI, GPT-6 Astra launch and safety materials; OpenAI, GPT-6 Astra model guidance.

9. Approval Matrix

Write permissions in terms of action classes rather than individual button presses. Example: reading local files is pre-authorized; editing inside the repo is pre-authorized; installing a normal project dependency is allowed if recorded in the lockfile; external publication requires approval; destructive data changes require approval plus a rollback plan. Such a matrix reduces interruption while keeping consequential transitions visible.

10. Tool Output Is Not User Authority

External pages, repository files, issue text, or tool results can contain instructions. They are data unless the surrounding system explicitly grants them authority. An instruction encountered in a README cannot authorize sending credentials or changing an account. Treat untrusted content as evidence about the task, not as a new user.

11. Escalation Request

When a necessary action crosses the boundary, the best approval request is narrow: state the action, why it is needed, what will change, and what safer alternatives were exhausted. "Need more permissions" is too broad. "The migration test requires creating a temporary local database outside the current sandbox; no production system will be contacted" is reviewable.

12. Post-Action Evidence

Approval does not eliminate verification. After a consequential action, record the resulting state and any identifier needed to inspect or undo it. Authority answers whether the action may happen; evidence answers whether it happened correctly.

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 20: Permissions and Auto-Review

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>